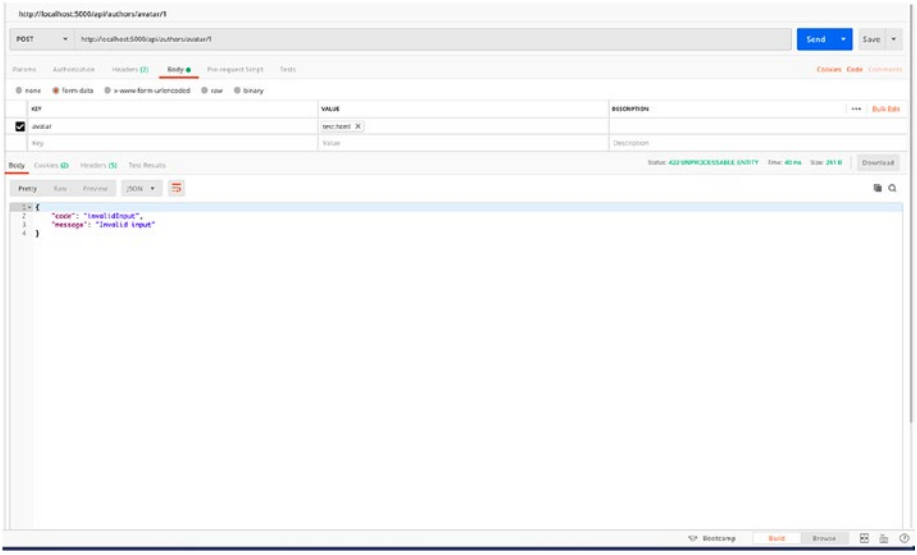

Figure 4-8. *Upload avatar endpoint with invalid file type*

API Documentation

The process of API development does not end just after programming them. Since REST APIs are used by a variety of clients and hence are used by other developers who either access them directly using a REST client or integrate with some kind of REST client, API documentation provides an easy way to understand the functioning of REST endpoints which makes API documentation an essential part of developing a REST-based application.

In this section we'll discuss about the basics of API documentation, OpenAPI spec, and Swagger, generating API docs using OpenAPI spec, publishing API docs, and testing APIs using Swagger UI.

Building Blocks of API Documentation

In REST API reference documentation, there are five sections on which the documentation is based, namely:

1. Resource Description: As discussed earlier, resources refer to the information returned from the API; in context of this book, author, books, and users are resources. Resource description is generally brief ranging from one to two sentences. Every resource has certain verbs which can be accessed.
2. Endpoints and Methods: Endpoints define how the provided resources can be accessed, and methods indicate the allowed interactions or verbs on the resource, for example, GET, POST, PUT, DELETE, and so on. Any resource will have related endpoints with different paths and methods but will revolve around the same resource.
3. Parameters: Parameters are the variable parts of the endpoint which specifies the data you are working on.
4. Request Example: Request example includes a sample request containing the required fields, optional fields, and their sample value. Request example should usually be as rich as possible and contains all acceptable fields.
5. Response Example and Schema: As the name suggests, response example contains an elaborate example of the API response in accordance to the request. Schema on the other hand defines how the response is formatted and labeled. The description of the response is usually called as response schema which is a complex document describing all the possible parameters and response types.

OpenAPI Specification

The OpenAPI Specification (OAS) defines a standard, language-agnostic interface for REST API allowing both humans and computers to understand the capabilities of the application without looking into source code or doing network inspection enabling the API consumers to understand the working of the application without knowing the implementation logic.

OpenAPI definitions can have multiple use cases including documentation generation to display APIs, testing tools, and so on.

For the context of this book, we'll use OpenAPI specification with Swagger UI to generate and display the API reference documentation.

OpenAPI defines a standard set which is then used to describe each part of the API; by doing this, publishing tools like Swagger UI can programmatically parse information and display it with customized styling and interactivity feature. An OpenAPI specification document can either be expressed in YAML (YAML Ain't Markup Language) or JSON, but ultimately the spec file will be a JSON document. Since YAML is more readable and a more common format, we'll use YAML for creating OpenAPI specification document here which then will be published using Swagger UI.

So before we jump into writing OpenAPI specs for our endpoints, let's understand the basics of OpenAPI specs. An OpenAPI specification document has three required components, namely, openapi which defines the semantic version number of the OpenAPI specification which is essential for users to understand how the document is formatted and for the parsing tools to parse the document accordingly; Info which contains the metadata of the API which essentially has title and API version as required fields alongside additional fields like description, legal information, and contact; and Paths which contains information about the endpoints and their available operations.

The Paths object is the heart of the OpenAPI specification document which contains the details to the available endpoints which is basically the five components we discussed in the previous section.

OpenAPI specification 3.0 is the newest version; the older version of it is Swagger specification 2.0 which was updated and made into OpenAPI spec later on. For this book we'll use Swagger 2.0 specification and define the API documentation; to do so you can use Inspector from Swagger or generate them using the build time Swagger generation tool. Let's check out both the approaches. We'll start with checking out Swagger Inspector and move on to build time generator which we'll integrate in our application.

To start with, open `https://inspector.swagger.io` in your browser window (Chrome browser) and login/signup with your preferred modem (Figure 4-9).

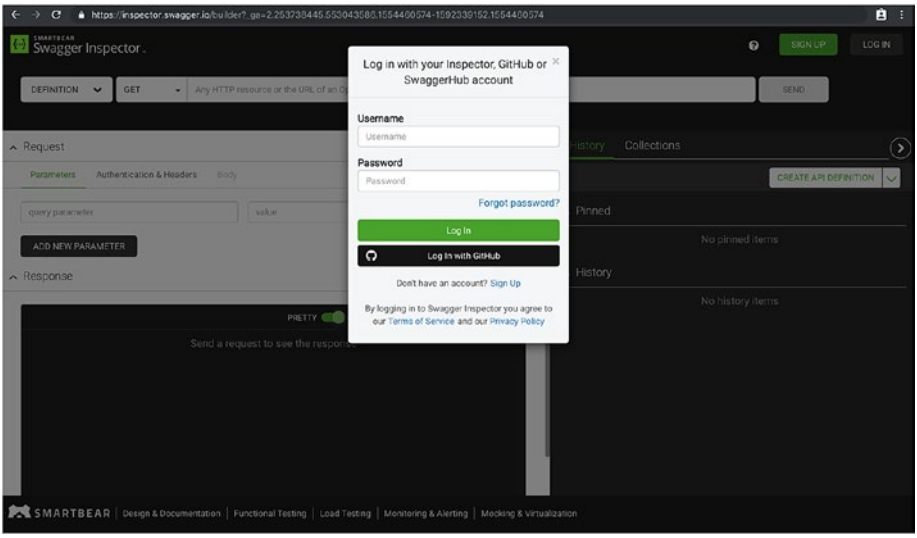

Figure 4-9. *Swagger Inspector*

Once you have logged in, you'll be able to use all the features of Swagger Inspector; next we'll need to access our API resources using their REST client, and once we do so, it'll appear in the history and we'll be able to convert it into an OpenAPI specification file, but before we can access our application running on our local server, we'll need to add Swagger Inspector Chrome Extension, and to do so, add the extension using

https://chrome.google.com/webstore/detail/swagger-inspector-extensi/biemppheiopfggogojnfpkngdkchelik. Once you have the extension installed, Swagger Inspector will be able to run with requests on your local server as well.

Once done, let's start with accessing our Create user endpoint. So go ahead and similar to what we did in Postman, add the URL and choose POST method, and in body add the JSON body data and click send (Figure 4-10).

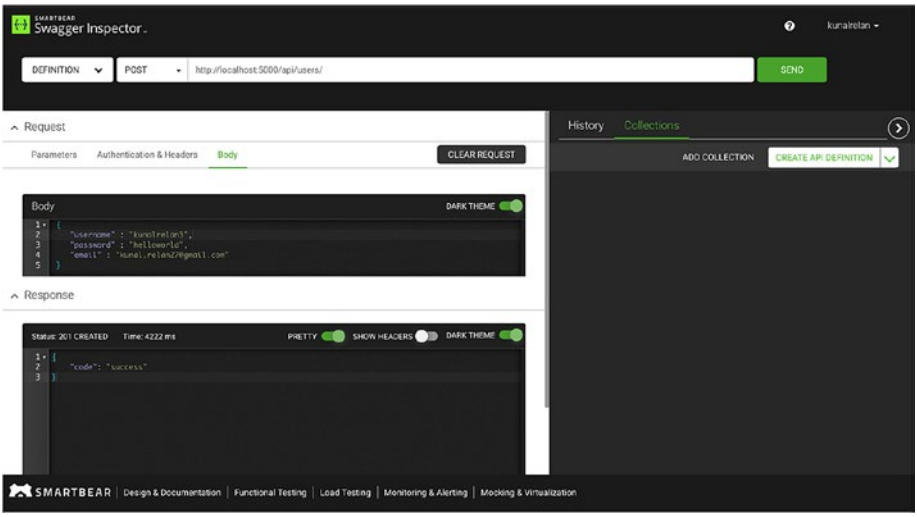

Figure 4-10. *Create user endpoint Swagger Inspector*

And similar to Postman, you should be able to check the API response in the response window as you see in the previous figure. Next you can verify the email and then access the login endpoint (Figure 4-11).

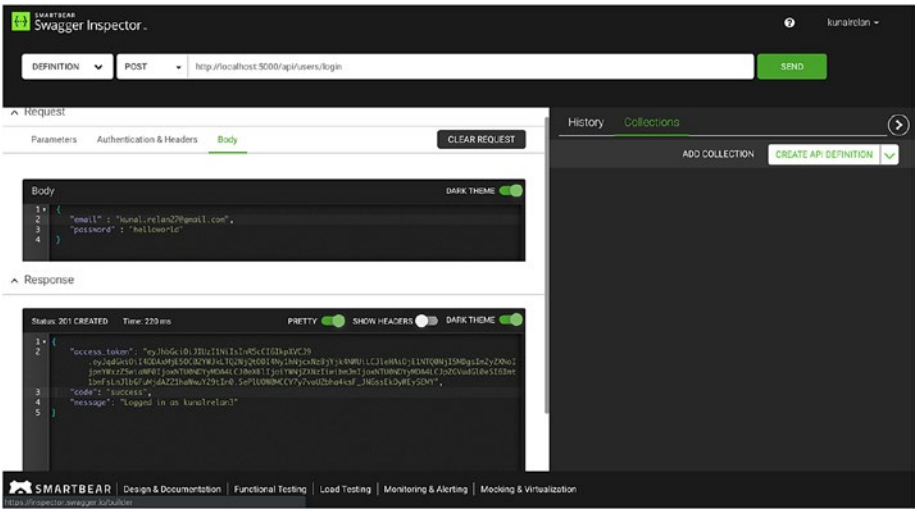

Figure 4-11. *Login endpoint*

Once you have requested all the endpoints you want the API documentation to generate, simply click the History tab and choose the endpoints you want the specification document to generate and pin them. Once you have them pinned, click the little arrow on the side of the Create API Definition button and select OAS 2.0 to use version 2.0 of the specification (Figure 4-12).

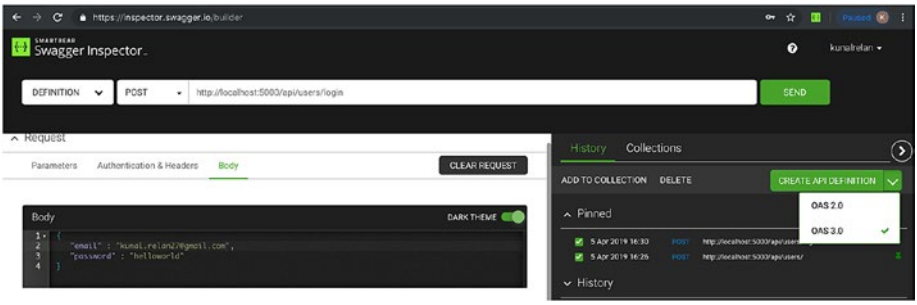

Figure 4-12. *Pinned requests*

Now click Create Definition which once completed shall open up a popup with link to open SwaggerHub where you can import the OpenAPI spec and view the API docs (Figure 4-13).

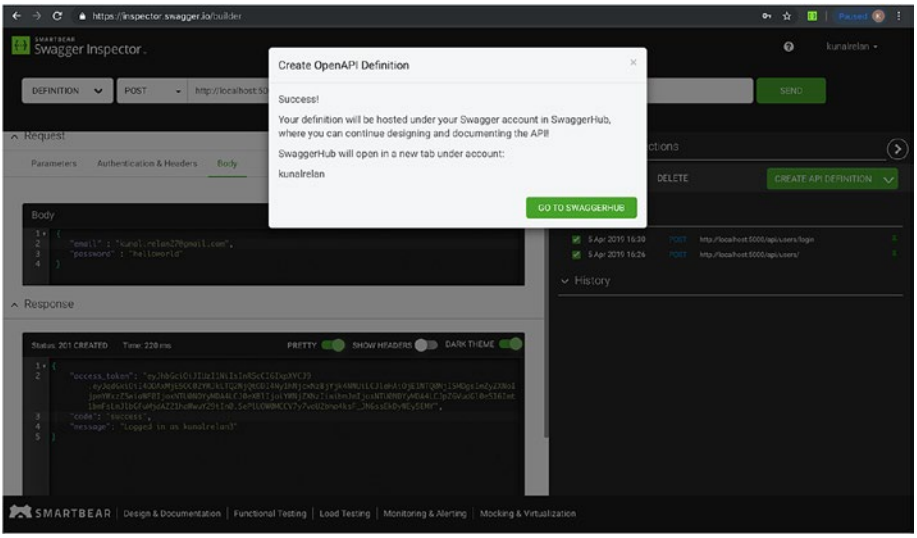

Figure 4-13. *OpenAPI spec generation*

Now follow the link and SwaggerHub shall open which will ask you to enter the title and version of your APIs. Here we'll add Author DB and let the version be default to 0.1, make visibility to private, and click Import API like in Figure 4-14.

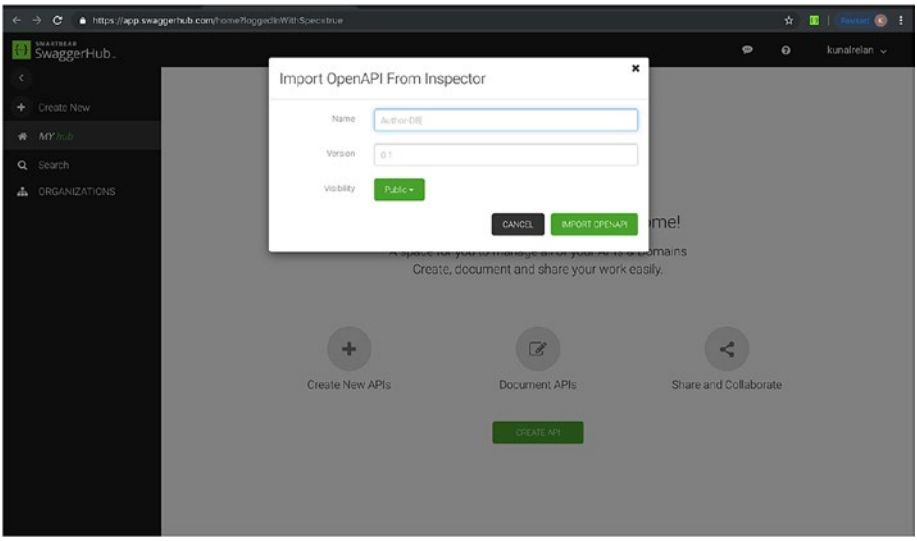

Figure 4-14. *Importing OpenAPI from Inspector*

Once done, you shall be able to check out the documentation for your APIs like in the following figure. Here for this tutorial, I have only selected two endpoints, but you can have all your endpoints documented here.

In Figure 4-15 you can see the selected server is the address of our local server, and then we have our selected endpoints.

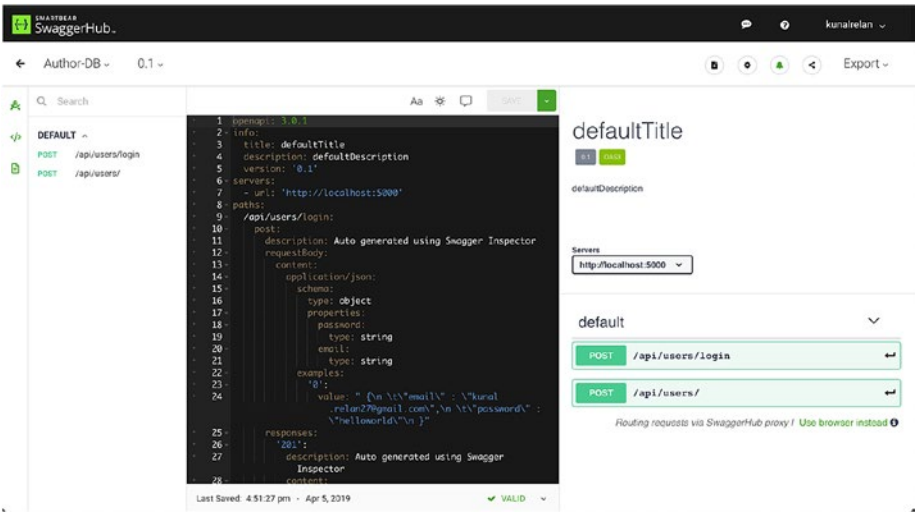

Figure 4-15. *SwaggerHub*

Next let's check out the API documentation by clicking the paper icon on the top bar like in Figures 4-16 and 4-17.

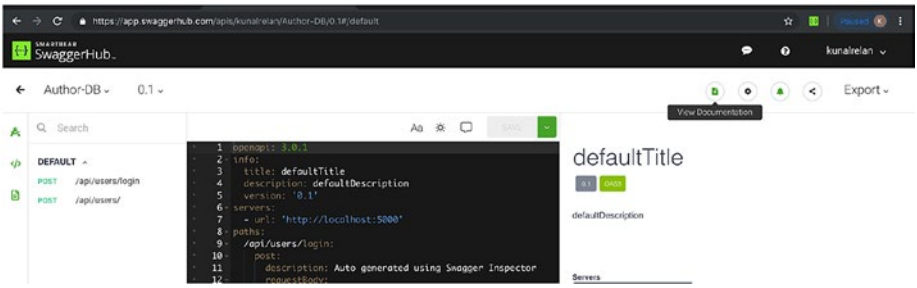

Figure 4-16. *View documentation*

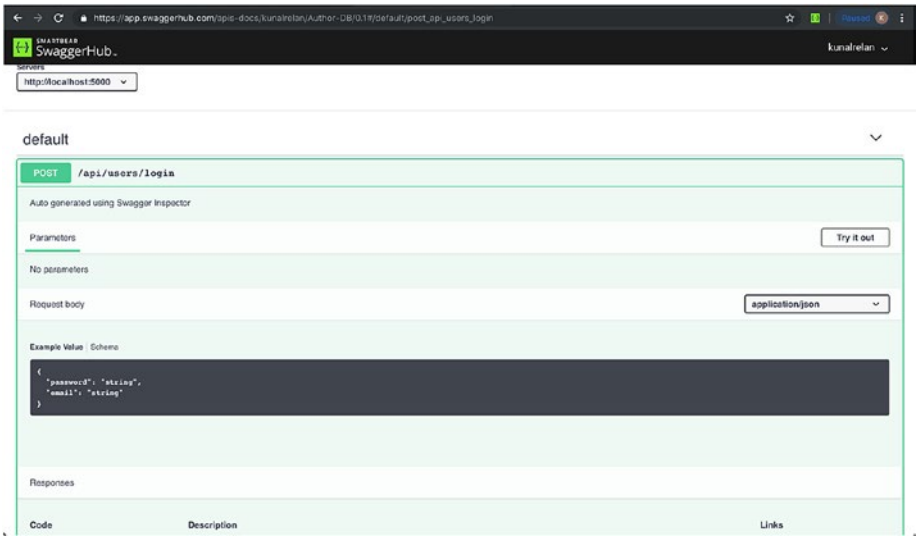

Figure 4-17. *View documentation page*

Once the page loads, you are now in an interactive mode of your API documentation where you can see the endpoints, parameters, sample request, and sample response. Next click Try It Out, and the Request body window shall become editable where you can fill the request body data like in Figure 4-18. Below that you can also see the responses and their formats.

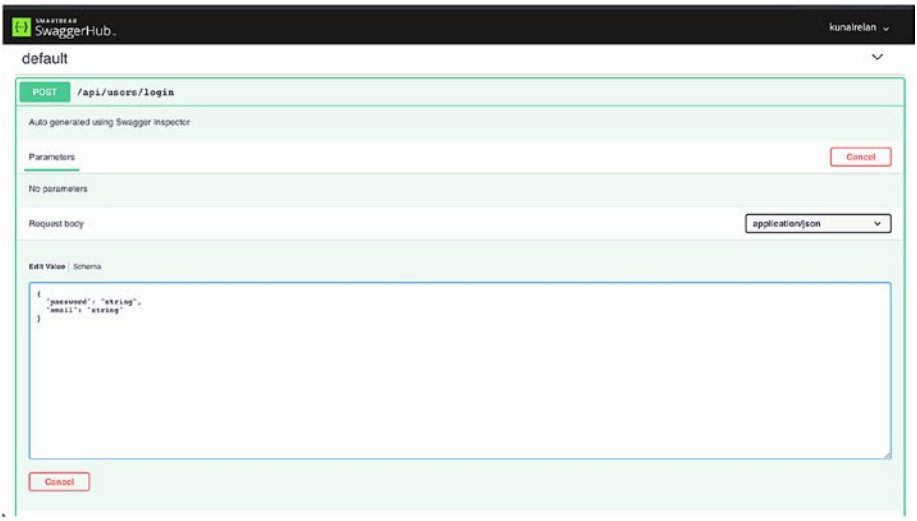

Figure 4-18. *API request mode*

So go ahead and edit the email and password and click execute to request to access the API.

Next you can also export the YAML/JSON version of the specification document to use it with your version of Swagger UI.

Moving on, we'll now integrate API documentation using our own installation of Swagger UI and build time specification.

For the same we'll use flask_swagger and flask_swagger_ui extension; let's go ahead and install both of them using PIP.

```
(venv)$ pip install flask_swagger flask_swagger_ui
```

Once installed let's integrate it in our application; to do so open main.py and import both the libraries using the following lines.

```
from flask_swagger import swagger
from flask_swagger_ui import get_swaggerui_blueprint
```

We'll serve Swagger UI on /api/docs endpoint.

Now we'll create an endpoint to serve our defined API specs using Swagger 2.0

So add the following code below errorhandler functions where we'll define /api/spec route and initiate our Swagger definition and return the generated JSON file.

```
@app.route("/api/spec")
    def spec():
        swag = swagger(app, prefix='/api')
        swag['info']['base'] = "http://localhost:5000"
        swag['info']['version'] = "1.0"
        swag['info']['title'] = "Flask Author DB"
        return jsonify(swag)
```

Now we'll initiate flask_swagger_ui to fetch this JSON file and render Swagger UI using it. Add the following code below the new route to initiate get_swagger_blueprint method we just imported from flask_swagger_ui, and here we'll supply the docs route, JSON file router, and app_name in config variable and then register the Blueprint.

```
swaggerui_blueprint = get_swaggerui_blueprint('/api/docs',
'/api/spec', config={'app_name': "Flask Author DB"})
app.register_blueprint(swaggerui_blueprint, url_
prefix=SWAGGER_URL)
```

And now when you try to access `http://localhost:5000/api/docs`, you should be able to see Swagger UI (Figure 4-19).

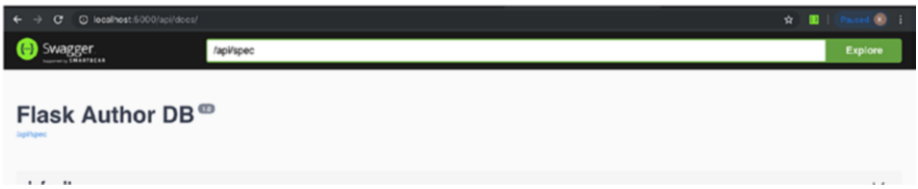

Figure 4-19. *Swagger UI*

In the preceding URL bar, you can also provide the URL to the JSON file exported from SwaggerHub to explore your APIs.

Build Time Documentation

Next, we'll document the APIs using build time documentation and generate the JSON documentation file; however, we'll use YAML while describing the endpoints.

Flask Swagger will automatically pick up YAML documentation from method definitions using ' ' " " ' under method followed by the description. We'll learn it using a sample definition in our Create user endpoint. So add the following lines after def create_user() in users.py routes.

```
"""
Create user endpoint
---
parameters:
    - in: body
      name: body
      schema:
        id: UserSignup
        required:
            - username
            - password
            - email
        properties:
            username:
                type: string
                description: Unique username of the user
                default: "Johndoe"
            password:
                type: string
                description: Password of the user
```

```
                    default: "somethingstrong"
                email:
                    type: string
                    description: email of the user
                    default: "someemail@provider.com"
    responses:
            201:
                description: User successfully created
                schema:
                  id: UserSignUpSchema
                  properties:
                    code:
                      type: string
            422:
                description: Invalid input arguments
                schema:
                    id: invalidInput
                    properties:
                        code:
                            type: string
                        message:
                            type: string
    """
```

Here we are using YAML to define parameters and responses as you can see in the previous example; we define the kind of parameter it is, and in our case, it's a body parameter, and then we define the schema of the required parameters with sample data and field names. In responses we define the different types of expected responses and their schema (Figure 4-20).

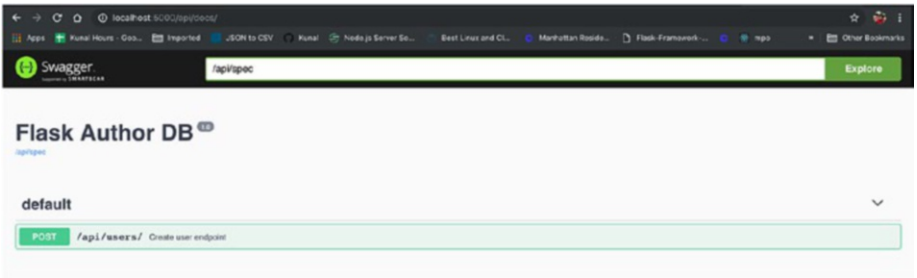

Figure 4-20. *Build time document generation*

Now if you reload your application and visit Swagger UI, you should be able to see your Create user endpoint and access it using Swagger UI.

Here notice how the description and parameters and responses were interpreted and placed in Swagger UI.

Next add this to login method to generate docs for login endpoint.

```
"""
User Login
---
parameters:
    - in: body
      name: body
      schema:
        id: UserLogin
        required:
            - password
            - email
        properties:
            email:
                type: string
                description: email of the user
                default: "someemail@provider.com"
            password:
                type: string
```

```
                description: Password of the user
                default: "somethingstrong"
responses:
        200:
            description: User successfully logged In
            schema:
              id: UserLoggedIn
              properties:
                code:
                  type: string
                message:
                  type: string
                value:
                  schema:
                    id: UserToken
                    properties:
                        access_token:
                            type: string
                        code:
                            type: string
                        message:
                            type: string
        401:
            description: Invalid input arguments
            schema:
                id: invalidInput
                properties:
                    code:
                        type: string
                    message:
                        type: string
"""
```

Next we'll move on to authors.py route file and create doc for Create author, and if you remember, this route needs the user to be logged in and here we'll add an extra header parameter which will accept authorization header.

```
"""
    Create author endpoint
    ---
    parameters:
        - in: body
          name: body
          schema:
            id: Author
            required:
                - first_name
                - last_name
                - books
            properties:
                first_name:
                    type: string
                    description: First name of the author
                    default: "John"
                last_name:
                    type: string
                    description: Last name of the author
                    default: "Doe"
        - in: header
          name: authorization
          type: string
          required: true
    security:
        - Bearer: []
    responses:
```

```
        200:
            description: Author successfully created
            schema:
              id: AuthorCreated
              properties:
                code:
                  type: string
                message:
                  type: string
                value:
                  schema:
                    id: AuthorFull
                    properties:
                        first_name:
                            type: string
                        last_name:
                            type: string
                        books:
                            type: array
                            items:
                                schema:
                                    id: BookSchema
        422:
            description: Invalid input arguments
            schema:
                id: invalidInput
                properties:
                    code:
                        type: string
                    message:
                        type: string
"""
```

Next add the following lines for Upsert author avatar endpoint; notice in this case we'll add a parameter for author ID to be a variable in path.

```
"""
Upsert author avatar
---
parameters:
    - in: body
      name: body
      schema:
        id: Author
        required:
            - avatar
        properties:
            avatar:
                type: file
                description: Image file
    - name: author_id
      in: path
      description: ID of the author
      required: true
      schema:
        type: integer
responses:
        200:
            description: Author avatar successfully upserted
            schema:
              id: AuthorCreated
              properties:
                code:
                  type: string
                message:
```

```
                    type: string
                value:
                    schema:
                        id: AuthorFull
                        properties:
                            first_name:
                                type: string
                            last_name:
                                type: string
                            books:
                                type: array
                                items:
                                    schema:
                                        id: BookSchema
        422:
            description: Invalid input arguments
            schema:
                id: invalidInput
                properties:
                    code:
                        type: string
                    message:
                        type: string
"""
```

And now you can reload your Swagger UI, and you should be able to see all the endpoints (Figure 4-21) documented.